

PROJECT BASED LEARNING REPORT

Airline Reservation & Flight Operations Management System



Team Members

Rathneshvaran.I - 25WU0101105
Lohit Sitharth.V.O - 25WU0101066
Mrithun Jagan - 25WU0101078
Sanjay.K - 25WU0101122

Class : CSE RHINOS

Specification: CSE CORE

Under Guidance of : Dr. Kiran Mayee Adavala

Department of Computer Science and Engineering

Academic Year

2026 –27

CONTENTS

1. Abstract
2. Introduction and Problem Statement
3. Objectives and Scope
4. Software and Hardware Requirements
5. ER Diagram
6. Relational Schema and Normalization
7. Data Dictionary
8. SQL Commands Used (DDL/DML) with Sample Outputs
9. Queries with Outputs
10. UI Design and Screenshots
11. Implementation Details
12. Testing (Test Cases and Results)
13. Conclusion and Future Enhancements
14. References
15. Contribution of Each Member
16. Appendix: GitHub Repository Link

1. ABSTRACT

SkyOps is a relational database management system designed to model the connected activities involved in airline reservation and flight operations. The system uses a normalized MySQL database and a Flask-based web interface to manage airports, routes, aircraft, seats, flights, passengers, fares, bookings, tickets, check-ins, baggage, cancellations and payments. The repository documents 13 major operational tables and provides schema, normalization, reporting queries, a data dictionary and systematic test cases.

The design focuses on data integrity and prevention of operational conflicts. Primary keys uniquely identify records, foreign keys preserve relationships, UNIQUE constraints prevent duplicate values, CHECK constraints enforce valid ranges and enumerations, and transactional seat/capacity validation protects the booking process. The application supports flight search, booking, booking lookup, check-in, baggage entry, cancellation/refund processing, passenger CRUD and operational reports.

The project demonstrates the complete DBMS workflow: requirements analysis, ER modelling, conversion to relational schema, normalization to Third Normal Form (3NF), SQL implementation, application connectivity, reporting and functional testing. The project also demonstrates synchronization between the Flask user interface and MySQL database through passenger CRUD operations.

Keywords: DBMS, MySQL, Flask, Airline Reservation, ER Model, Normalization, 3NF, SQL, CRUD, Database Integrity.

2. INTRODUCTION AND PROBLEM STATEMENT

2.1 Introduction

Airline reservation systems contain highly connected records. A single passenger interaction can involve a flight, route, aircraft, seat, fare, booking, ticket, payment, check-in and baggage record. If these records are maintained without a structured relational model, duplicated or inconsistent data can result in booking conflicts, invalid check-ins, incorrect passenger manifests and unreliable operational reports.

SkyOps addresses these issues by organizing airline operations into related entities and enforcing integrity at the database and application levels. The booking entity acts as a central link between the passenger, flight and fare, while ticket, check-in and baggage records connect reservation activities to airport operations.

The project uses MySQL for data storage and Python with Flask for the application layer. The web interface provides access to the implemented database operations.

2.2 Problem Statement

The problem addressed by this project is the need for a centralized relational database system that maintains consistent airline reservation and flight-operation records, prevents invalid or duplicate bookings, supports operational transactions and exposes database functionality through a working web interface.

Without a structured database, the following problems may occur:

Duplicate passenger and booking records.

Seat allocation conflicts.

Incorrect passenger manifests.

Inconsistent baggage information.

Invalid check-in operations.

Incorrect cancellation and refund records.

Difficulty generating operational reports.

Data redundancy and modification anomalies.

SkyOps addresses these problems through relational modelling, normalization, database constraints and application-level validation.

2.3 Proposed Solution

The proposed solution consists of a MySQL database containing 13 related tables and a Flask web application for interacting with the database.

The database organizes airline data into separate entities, while primary keys and foreign keys maintain relationships. Application logic validates booking operations, checks seat availability and aircraft capacity, processes check-in requests and records cancellations.

The workflow connects flight setup, reservation and payment with airport operations. Check-in and baggage records can be traced back to the relevant ticket, booking and flight.

3. OBJECTIVES AND SCOPE

3.1 Objectives

The main objectives of the project are:

1. Design a normalized relational database.
2. Maintain data integrity across airline operations.
3. Prevent duplicate or invalid bookings.
4. Support flight search and reservation.
5. Manage passengers, tickets, check-ins and baggage.
6. Process cancellations and payments.
7. Generate operational reports.
8. Connect the database to a working frontend.

3.2 Functional Scope

The system supports the following activities:

Dashboard and flight information viewing.

Flight search based on origin, destination and date.

Flight booking and seat allocation.

Booking lookup.

Passenger management.

Online check-in.

Baggage entry.

Cancellation and refund processing.

Operational reporting.

3.3 Database Scope

The database contains the following 13 operational tables:

- airport
- route
- aircraft
- seat
- flight
- passenger
- fare
- booking
- ticket
- checkin
- baggage
- cancellation
- Payment

3.4 Intended Users

The intended users include airline administrators, reservation staff, check-in staff, operations staff and passengers.

The application is an academic prototype designed to demonstrate the database and its connected operations.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

Component	Requirement
-----------	-------------

Programming language	Python 3
Web framework	Flask 3.1.2
Database	MySQL 9.x
Database connector	mysql-connector-python 9.4.0
Environment configuration	python-dotenv 1.1.1
Frontend	HTML, CSS and JavaScript
Template engine	Jinja2
Browser	Chrome, Edge, Firefox or another modern browser
Source control	Git and GitHub
Code editor	Visual Studio Code

4.2 Hardware Requirements

The repository does not prescribe a minimum hardware specification. The following is a practical recommended environment for classroom demonstration and local development.

Component	Recommended Requirement
Processor	Dual-core or better processor
RAM	4 GB or more
Storage	Approximately 2 GB free space for project tools
Network	Internet connection when cloning from GitHub
Input devices	Keyboard and mouse or trackpad
Display	Standard laptop or desktop display

The application runs locally at <http://127.0.0.1:5000> in the documented setup.

4.3 Execution Requirements

The application requires Python, MySQL Server and the required Python packages. The repository includes a README describing the setup procedure.

The application initialization routine creates the `airline_reservation` database, applies the schema, ensures indexes and loads demonstration data.

5. ER DIAGRAM

5.1 Overview

The Entity–Relationship model contains 13 entities, with one principal table representing each operational entity. Booking is the central transaction entity linking passenger, flight and fare.

Primary keys and foreign keys represent relationships between entities, with mostly one-to-many cardinalities and an optional cancellation relationship.

5.2 Entities

The major entities are:

AIRPORT

ROUTE

AIRCRAFT

SEAT

FLIGHT

PASSENGER

FARE

BOOKING

TICKET

PAYMENT

CHECKIN

BAGGAGE

CANCELLATION

5.3 Important Relationship Paths

The main relationship paths are:

Airport → Route → Flight

Flight → Aircraft → Seat

Passenger → Booking → Ticket

Ticket → Check-in

Ticket → Baggage

Booking → Payment

Booking → Cancellation

These relationships allow reservation records to be traced into flight operations.

5.4 Cardinality

One-to-Many (1): One record in an entity can be associated with multiple records in another entity. For example, one passenger can make multiple bookings.

One-to-One (1:1): One record in an entity is associated with one record in another entity under the intended relationship.

Optional Relationship (0..1): A booking may have no cancellation record or one cancellation record.

5.5 ER Diagram Image

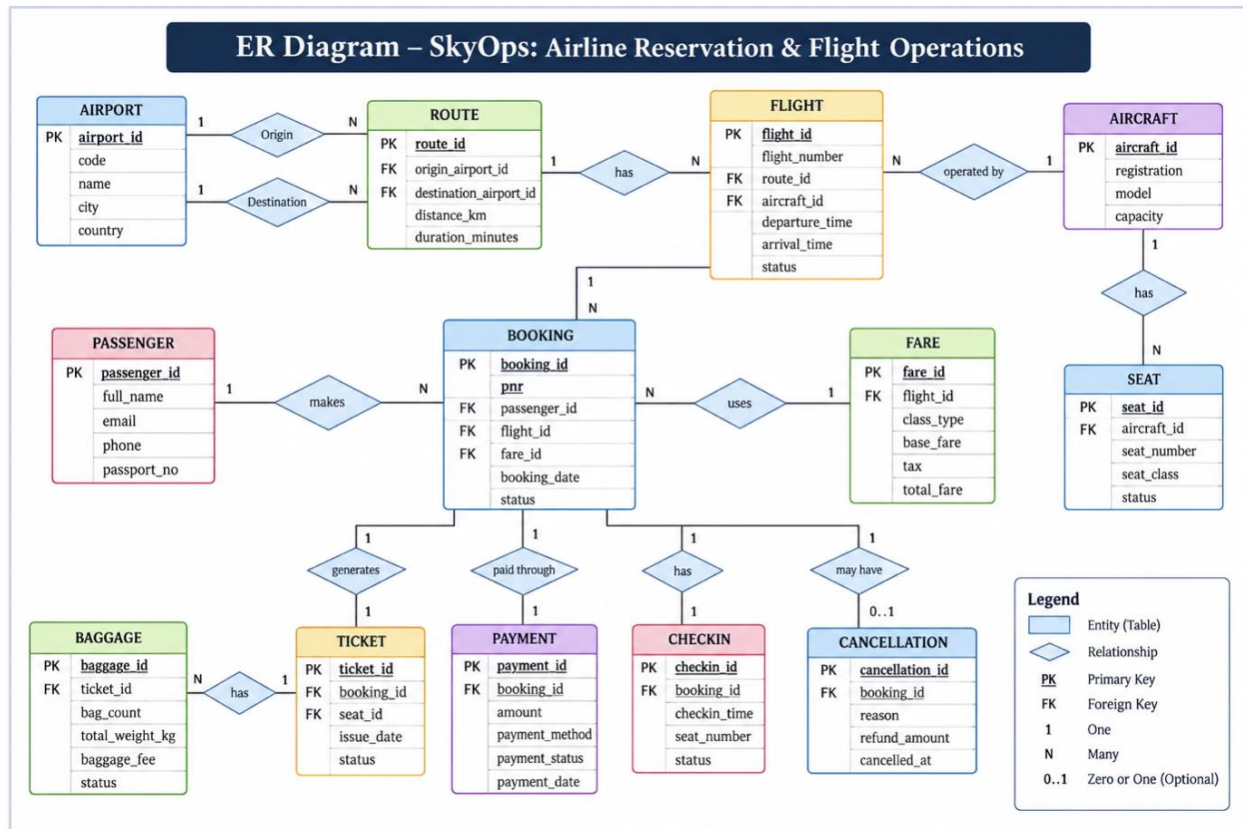


Figure 1: ER Diagram of the SkyOps Airline Reservation and Flight Operations Management System.

6. RELATIONAL SCHEMA AND NORMALIZATION

6.1 Relational Schema

The ER model is converted into relational tables. Each table represents a logical entity, and primary keys uniquely identify records while foreign keys maintain relationships.

The principal relations are summarized below.

```
AIRPORT(  airport_id PK,
          code,
          name,
          city,
          country
)
```

```
ROUTE(
  route_id PK,
```

```
    origin_airport_id FK,  
    destination_airport_id FK,  
    distance_km,  
    duration_minutes  
)
```

```
AIRCRAFT(  
    aircraft_id PK,  
    registration,  
    model,  
    capacity  
)
```

```
SEAT(  
    seat_id PK,  
    aircraft_id FK,  
    seat_number,  
    seat_class  
)
```

```
FLIGHT(  
    flight_id PK,  
    flight_number,  
    route_id FK,  
    aircraft_id FK,  
    departure_time,  
    arrival_time,  
    status  
)
```

```
PASSENGER(  
    passenger_id PK,  
    full_name,  
    email,  
    phone,  
    passport_no  
)
```

```
FARE(  
    fare_id PK,
```

```
    flight_id FK,  
    fare_class,  
    base_fare,  
    tax  
)
```

```
BOOKING(  
    booking_id PK,  
    pnr,  
    passenger_id FK,  
    flight_id FK,  
    fare_id FK,  
    booking_date,  
    status  
)
```

```
TICKET(  
    ticket_id PK,  
    ticket_number,  
    booking_id FK,  
    seat_id FK,  
    issued_at  
)
```

```
CHECKIN(  
    checkin_id PK,  
    ticket_id FK,  
    checkin_time,  
    status  
)
```

```
BAGGAGE(  
    baggage_id PK,  
    ticket_id FK,  
    bag_count,  
    total_weight_kg,  
    baggage_fee  
)
```

```
CANCELLATION(  
    cancellation_id PK,  
    ticket_id FK,  
    booking_id FK,  
    cancellation_date,  
    status  
)
```

```
cancellation_id PK,  
booking_id FK,  
cancelled_at,  
reason,  
refund_amount  
)
```

```
PAYMENT(  
  payment_id PK,  
  booking_id FK,  
  amount,  
  method,  
  payment_status,  
  transaction_ref,  
  paid_at  
)
```

The exact column definitions, data types and constraints are specified in the project's schema.sql file.

6.2 First Normal Form (1NF)

A relation satisfies First Normal Form when each attribute contains an atomic value and repeating groups are eliminated.

In SkyOps, passenger details, flight details and booking details are stored as separate attributes and related records instead of combining unrelated information into a single field.

6.3 Second Normal Form (2NF)

A relation satisfies Second Normal Form when it is in 1NF and every non-key attribute depends on the whole candidate key rather than only part of a composite candidate key.

SkyOps uses separate entities for passengers, flights, aircraft and bookings to represent their distinct information.

6.4 Third Normal Form (3NF)

A relation satisfies Third Normal Form when it is in 2NF and non-key attributes do not have inappropriate transitive dependencies on the primary key.

For example, aircraft model and capacity depend on aircraft identity. These attributes are stored in the AIRCRAFT relation and referenced from FLIGHT through aircraft_id.

This avoids repeatedly storing aircraft descriptive information in every flight record.

6.5 Benefits of Normalization

Normalization helps to:

Reduce data redundancy.

Improve data consistency.

Simplify updates to master information.

Prevent insertion anomalies.

Prevent update anomalies.

Prevent deletion anomalies.

Maintain a clear separation between logical entities.

The project uses 3NF as a practical normalization target for maintaining a structured relational database.

6.6 Relational Schema Image

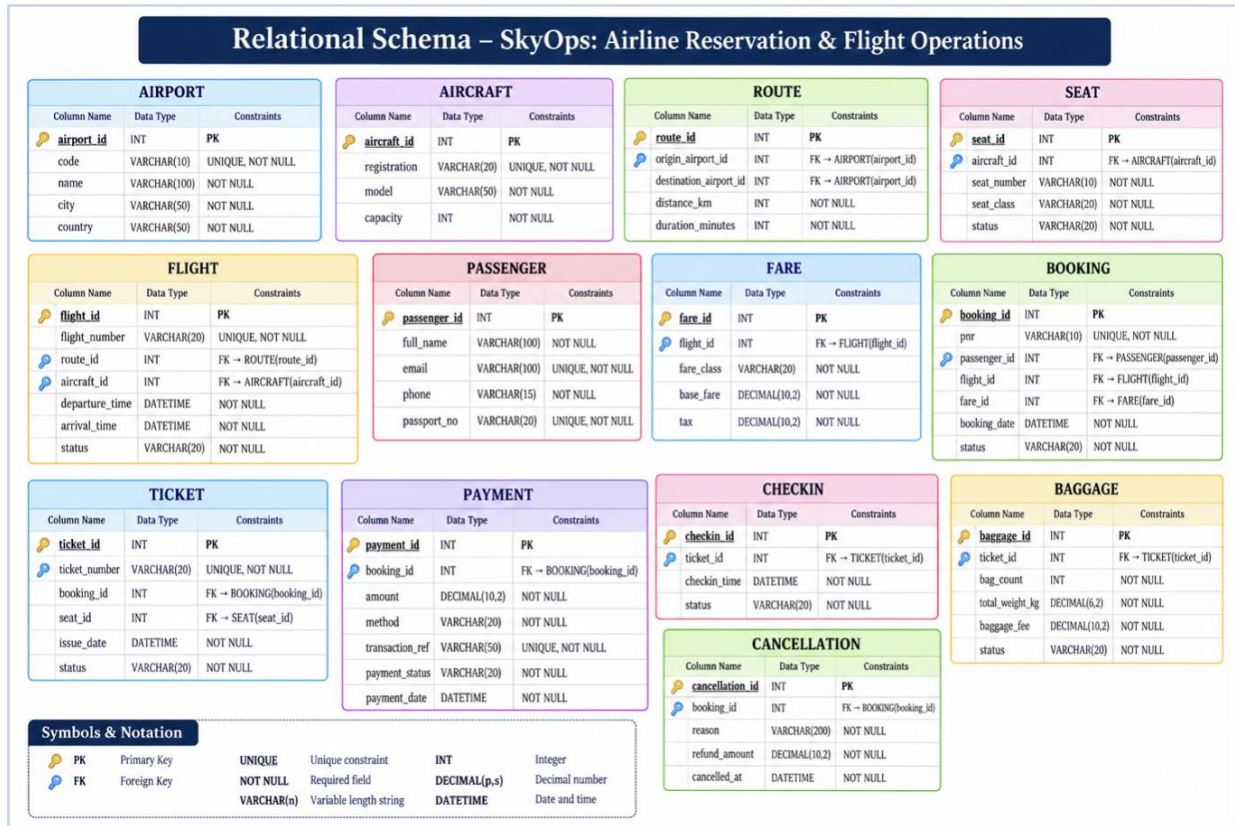


Figure 2: Relational Schema of the SkyOps Database.

7. DATA DICTIONARY

The data dictionary summarizes the table purposes, attributes, data types and principal constraints used in the database.

7.1 AIRPORT

Attribute	Data Type	Constraint/Purpose
-----------	-----------	--------------------

airport_id	INT	Primary key
code	VARCHAR(10)	Unique and not null
name	VARCHAR(150)	Not null
city	VARCHAR(80)	Not null
country	VARCHAR(80)	Not null

Purpose: Stores airport identification and location information.

7.2 ROUTE

Attribute	Data Type	Constraint/Purpose
route_id	INT	Primary key
origin_airport_id	INT	Foreign key to airport
destination_airport_id	INT	Foreign key to airport
distance_km	INT	Positive route distance
duration_minutes	INT	Positive route duration

Purpose: Represents routes between origin and destination airports.

7.3 AIRCRAFT

Attribute	Data Type	Constraint/Purpose
aircraft_id	INT	Primary key

registration	VARCHAR(30)	Unique registration
model	VARCHAR(100)	Aircraft model
capacity	INT	Positive capacity

Purpose: Stores aircraft details and seating capacity.

7.4 SEAT

Attribute	Data Type	Constraint/Purpose
seat_id	INT	Primary key
aircraft_id	INT	Foreign key to aircraft
seat_number	VARCHAR(10)	Seat identifier
seat_class	ENUM	Seat class
status	VARCHAR/ENUM	Seat status, where defined

Purpose: Stores seat information associated with an aircraft.

7.5 FLIGHT

Attribute	Data Type	Constraint/Purpose
flight_id	INT	Primary key
flight_number	VARCHAR(20)	Flight identifier
route_id	INT	Foreign key to route
aircraft_id	INT	Foreign key to aircraft

departure_time	DATETIME	Departure timestamp
arrival_time	DATETIME	Arrival timestamp
status	ENUM	Flight status

Purpose: Stores scheduled flight information.

7.6 PASSENGER

Attribute	Data Type	Constraint/Purpose
passenger_id	INT	Primary key
full_name	VARCHAR(120)	Passenger name
email	VARCHAR(150)	Unique email
phone	VARCHAR(30)	Contact number
passport_no	VARCHAR(40)	Unique when supplied

Purpose: Stores passenger identity and contact information.

7.7 FARE

Attribute	Data Type	Constraint/Purpose
fare_id	INT	Primary key
flight_id	INT	Foreign key to flight
fare_class	ENUM	Fare class
base_fare	DECIMAL(10,2)	Base fare
tax	DECIMAL(10,2)	Tax amount

Purpose: Stores fare information associated with flights.

7.8 BOOKING

Attribute	Data Type	Constraint/Purpose
-----------	-----------	--------------------

booking_id	INT	Primary key
pnr	VARCHAR(20)	Unique booking reference
passenger_id	INT	Foreign key to passenger
flight_id	INT	Foreign key to flight
fare_id	INT	Foreign key to fare
booking_date	DATETIME	Booking timestamp
status	ENUM	Booking status

Purpose: Stores passenger reservations and links passengers, flights and fares.

7.9 TICKET

Attribute	Data Type	Constraint/Purpose
ticket_id	INT	Primary key
ticket_number	VARCHAR(30)	Unique ticket number
booking_id	INT	Unique foreign key to booking
seat_id	INT	Foreign key to seat
issued_at	DATETIME	Ticket issue timestamp

Purpose: Stores ticket information associated with a booking.

7.10 CHECKIN

Attribute	Data Type	Constraint/Purpose
checkin_id	INT	Primary key
ticket_id	INT	Unique foreign key to ticket
checkin_time	DATETIME	Check-in timestamp

status	ENUM	Check-in status
--------	------	-----------------

Purpose: Records passenger check-in information.

7.11 BAGGAGE

Attribute	Data Type	Constraint/Purpose
baggage_id	INT	Primary key
ticket_id	INT	Foreign key to ticket
bag_count	INT	Non-negative bag count
total_weight_kg	DECIMAL(8,2)	Non-negative total weight
baggage_fee	DECIMAL(10,2)	Non-negative fee

Purpose: Stores baggage quantities, weight and charges.

7.12 CANCELLATION

Attribute	Data Type	Constraint/Purpose
cancellation_id	INT	Primary key
booking_id	INT	Unique foreign key to booking
cancelled_at	DATETIME	Cancellation timestamp
reason	VARCHAR(255)	Cancellation reason
refund_amount	DECIMAL(10,2)	Non-negative refund

Purpose: Records booking cancellation and refund information.

7.13 PAYMENT

Attribute	Data Type	Constraint/Purpose
payment_id	INT	Primary key
booking_id	INT	Foreign key to booking
amount	DECIMAL(10,2)	Positive amount
method	ENUM	Payment method
payment_status	ENUM	Payment status
transaction_ref	VARCHAR(40)	Unique transaction reference
paid_at	DATETIME	Payment timestamp

Purpose: Stores payment transactions associated with bookings.

7.14 Integrity Rules

The database uses integrity rules such as:

- Unique airport codes.
- Unique aircraft registrations.
- Unique passenger emails and supplied passport numbers.
- Unique booking PNRs.
- Unique ticket numbers.
- Unique seats within an aircraft.
- Unique fare classes per flight.
- Non-negative baggage, fare and refund values.
- Transactional validation of seat allocation and aircraft capacity.
- The actual database definitions in schema.sql are authoritative for the final implementation.

8. SQL COMMANDS USED (DDL AND DML) WITH SAMPLE OUTPUTS

SQL is used to define database structures, insert and retrieve records, and modify stored information.

8.1 DDL: Create a Table

The following example demonstrates the structure of a passenger table.

```
CREATE TABLE passenger (  
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,  
    full_name VARCHAR(120) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    phone VARCHAR(30) NOT NULL,  
    passport_no VARCHAR(40) UNIQUE  
) ENGINE=InnoDB;
```

The complete schema defines 13 tables and indexes for frequently used flight, booking and ticket columns. The schema uses InnoDB and applies primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, ENUM and DEFAULT constraints where defined.

Expected result: The table is created successfully if it does not already exist and all referenced objects are valid.

8.2 DML: Insert a Passenger

```
INSERT INTO passenger(full_name, email, phone)  
VALUES (  
    'Demo Passenger',  
    'demo@gmail.com',  
    '9876543210'  
);
```

Purpose: Inserts a passenger record into the database.

Expected output: One row is inserted if the email is not already present and all constraints are satisfied.

8.3 DML: Select a Passenger

```
SELECT *  
FROM passenger  
WHERE email = 'demo@gmail.com';
```

Purpose: Retrieves the passenger record matching the specified email.

8.4 DML: Update a Passenger

```
UPDATE passenger  
SET phone = '9876500000'  
WHERE email = 'demo@gmail.com';
```

Purpose: Updates the contact number of the specified passenger.

8.5 DML: Delete an Eligible Passenger

```
DELETE FROM passenger  
WHERE email = 'demo@gmail.com';
```

Purpose: Deletes the specified passenger if the database constraints and application rules permit it.

The application protects passengers who have existing bookings from deletion.

8.6 Passenger CRUD Demonstration

Operation	Result
INSERT	Passenger appears in the UI and is inserted into MySQL.
SELECT / VIEW	Passenger Records loads records from the MySQL passenger table.
UPDATE	Passenger information can be modified using an SQL UPDATE operation.
DELETE	A permitted demo passenger is removed from the UI and database.

9. QUERIES WITH OUTPUTS

The project includes joins, aggregate functions, nested queries and a reporting view. These queries demonstrate how SQL can retrieve useful information from related airline records.

9.1 Query 1: Flight Manifest

```

SELECT
    f.flight_number,
    oa.code AS origin,
    da.code AS destination,
    p.full_name,
    t.ticket_number,
    s.seat_number,
    COALESCE(ci.status, 'Not Checked-in') AS checkin
FROM booking b
JOIN passenger p
    ON p.passenger_id = b.passenger_id
JOIN flight f
    ON f.flight_id = b.flight_id
JOIN route r
    ON r.route_id = f.route_id
JOIN airport oa
    ON oa.airport_id = r.origin_airport_id
JOIN airport da
    ON da.airport_id = r.destination_airport_id
JOIN ticket t
    ON t.booking_id = b.booking_id
JOIN seat s
    ON s.seat_id = t.seat_id
LEFT JOIN checkin ci
    ON ci.ticket_id = t.ticket_id;

```

Purpose: Retrieves the flight number, origin, destination, passenger name, ticket number, seat and check-in status.

Representative output:

Flight	Origin	Destination	Passenger	Seat	Check-in
AI101	MAA	HYD	Rathnesh Ilangovan	1A	Not Checked-in
AI101	MAA	HYD	Aarav Sharma	2B	Not Checked-in
AI101	MAA	HYD	Priya Nair	3C	Not Checked-in

These are representative rows from the documented demonstration data. Actual output depends on the current database state.

9.2 Query 2: Flight Occupancy

```

SELECT
    f.flight_number,
    a.capacity,
    COUNT(
        CASE
            WHEN b.status = 'Confirmed' THEN 1
        END
    ) AS booked,
    ROUND(
        100.0 * COUNT(
            CASE
                WHEN b.status = 'Confirmed' THEN 1
            END
        ) / a.capacity,
        2
    ) AS occupancy_pct
FROM flight f
JOIN aircraft a
    ON a.aircraft_id = f.aircraft_id
LEFT JOIN booking b
    ON b.flight_id = f.flight_id
GROUP BY
    f.flight_id,
    f.flight_number,
    a.capacity;

```

Purpose: Calculates the number of confirmed bookings and occupancy percentage for each flight.

Representative output:

Flight	Capacity	Booked	Occupancy
AI101	180	3	1.67%

AI202	189	0	0.00%
6E303	220	0	0.00%
6E404	180	0	0.00%
UK505	189	0	0.00%
AI606	180	0	0.00%

9.3 Query 3: Flights Above a Booking Threshold

```

SELECT *
FROM flight
WHERE flight_id IN (
    SELECT flight_id
    FROM booking
    GROUP BY flight_id
    HAVING COUNT(*) > 2
);

```

Purpose: Uses a nested query to identify flights with more than two booking records.

Representative result: AI101 is returned because the documented demonstration data contains three confirmed bookings for that flight.

9.4 Other Project Reports

Report	Purpose
Route demand	Counts bookings by origin-destination route.

Baggage	Summarizes bag count, total weight and baggage fees by flight.
Cancellation/refund	Lists PNR, passenger, flight, cancellation reason and refund amount.
Revenue	Reports paid and refunded amounts by flight.
Flight summary view	Combines flight, route, airport, aircraft and confirmed-booking counts.

The repository also provides a view named `vw_flight_summary` for consolidated flight-level reporting.

[INSERT ACTUAL SQL QUERY AND OUTPUT SCREENSHOTS HERE]

10. UI DESIGN AND SCREENSHOTS

10.1 Overview

The application uses a Flask web interface connected to MySQL. The interface provides access to the airline reservation database through pages for dashboard viewing, flight search, passenger management, online check-in and baggage entry.

The pages read from or write to the `airline_reservation` database.

10.2 Dashboard

The dashboard presents flight and operational information to help users access the main system functions.

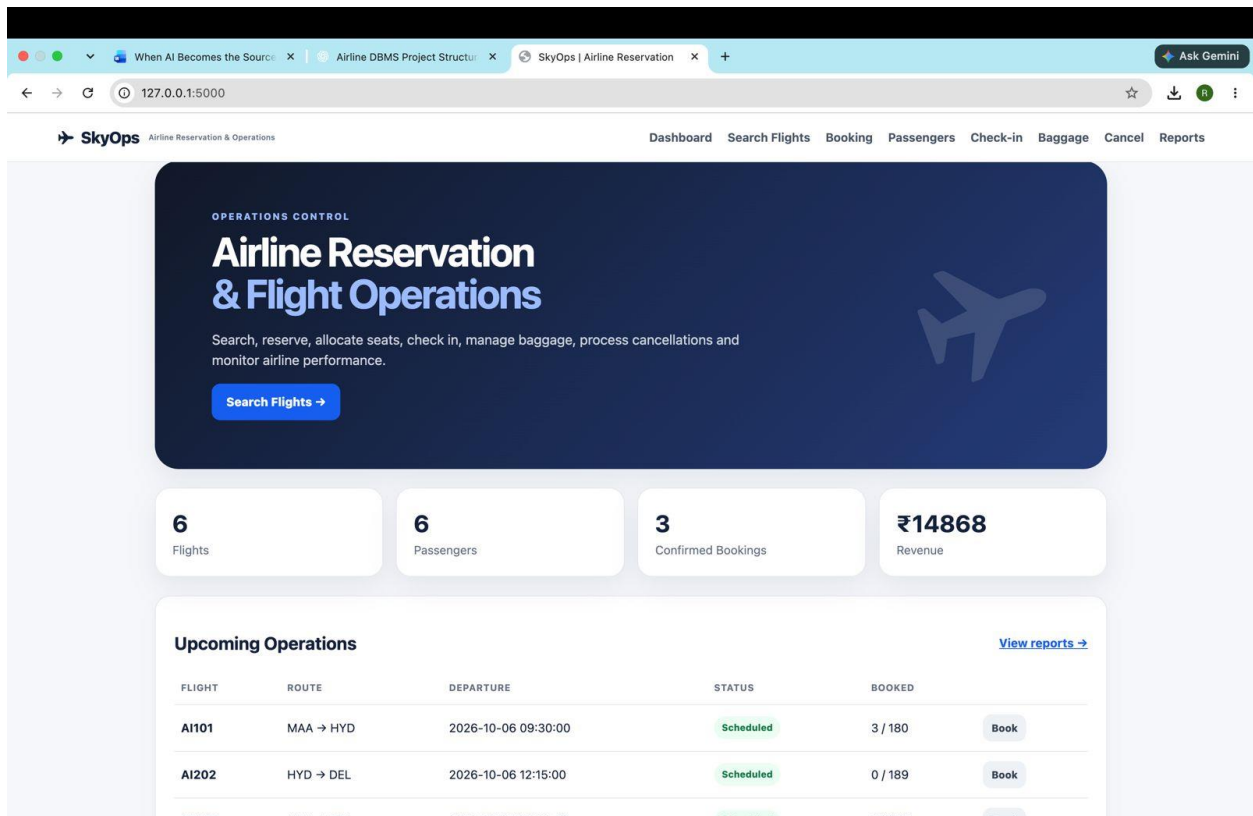


Figure 3: SkyOps Dashboard.

10.3 Flight Search

The Flight Search page filters flights by origin, destination and date.

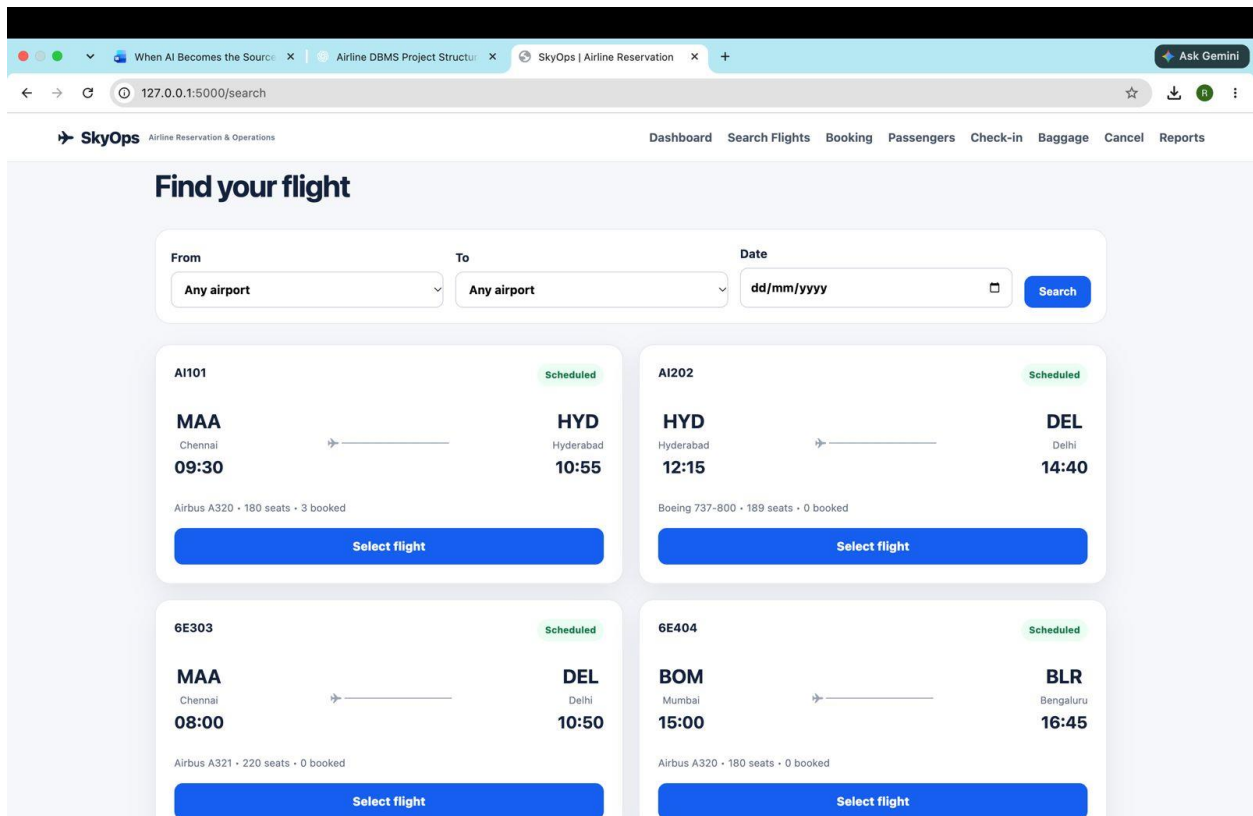


Figure 4: Flight Search Interface.

10.4 Passenger Management

The Passenger Management page supports passenger insertion, viewing and deletion of eligible records.

When AI Becomes the Source x | Airline DBMS Project Structur x | SkyOps | Airline Reservation x +

127.0.0.1:5000/passengers

Ask Gemini

SkyOps Airline Reservation & Operations

DashboardSearch FlightsBookingPassengersCheck-inBaggageCancelReports

DATABASE CRUD

Passenger Management

Add, view and delete passenger records directly through the Flask UI. Every action is stored in the MySQL `passenger` table.

INSERT

Add Passenger

MySQL INSERT

Full Name

Demo Passenger

Email

demo@gmail.com

Phone

9876543210

Passport No.
(optional)

P9999999

+ Add Passenger

VIEW

Passenger Records

6 records shown

ID	FULL NAME	EMAIL	PHONE	PASSPORT	BOOKINGS	ACTION
7	Lohit	lohit@gmail.com	9947567893	P2345676	0	Delete
6	Test Passenger	testdemo@gmail.com	9876543210	—	1	Booked

Figure 5: Passenger Management Interface.

10.5 Online Check-in

The Online Check-in page retrieves a booking and creates a check-in record for a confirmed booking, subject to validation.

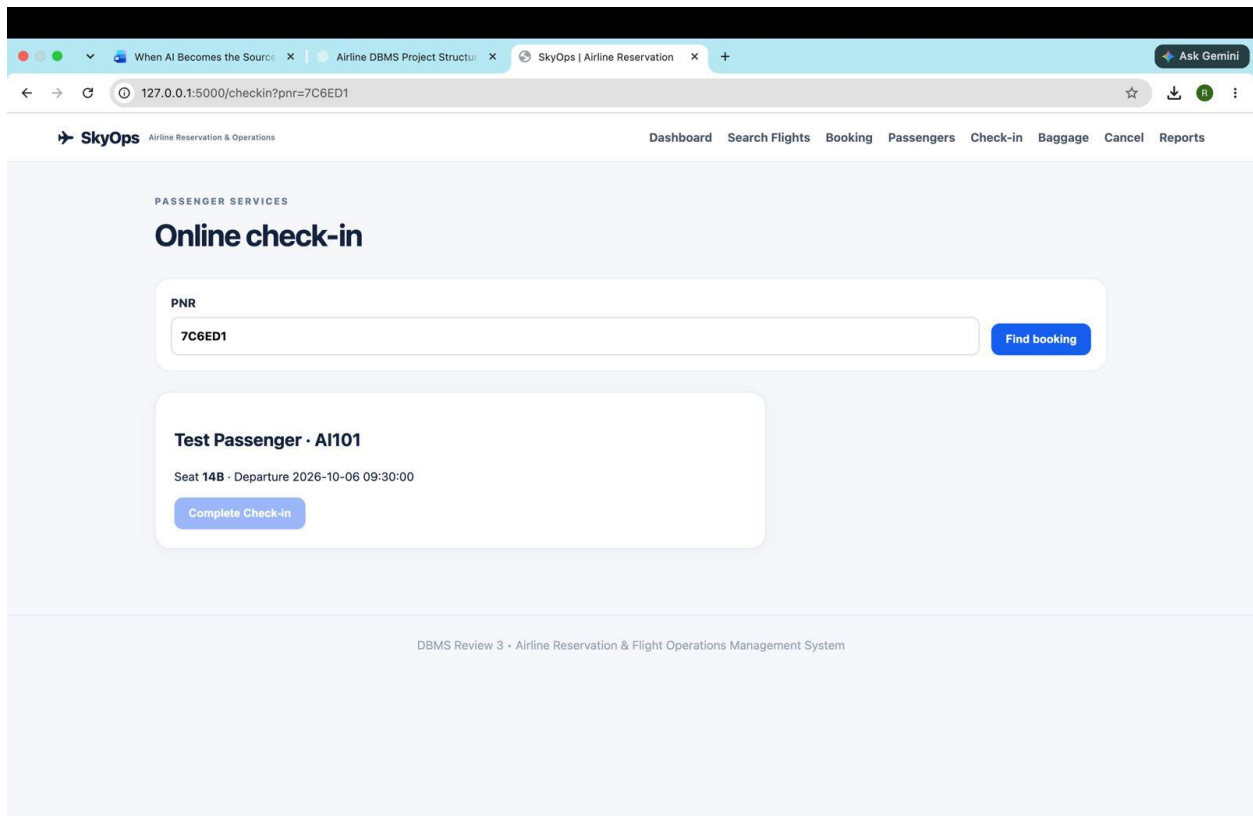


Figure 6: Online Check-in Interface.

10.6 Baggage Entry

The Baggage Entry page records the bag count, total weight and applicable baggage fee.

Baggage recorded. Fee: ₹0.00

BAGGAGE SERVICES

Baggage entry

PNR

7C6ED1 [Find ticket](#)

Test Passenger · AI101 · Seat 14B

Bags	Total weight (kg)	
1	10	Record baggage

First 15 kg included in fare; excess is ₹300/kg for this demo.

DBMS Review 3 · Airline Reservation & Flight Operations Management System

Figure 7: Baggage Management Interface.

10.7 Database Synchronization

The application demonstrates interaction between the Flask interface and MySQL database.

When a passenger is added through the UI, the record is inserted into MySQL. When an eligible passenger is deleted, the corresponding record is removed from the database.

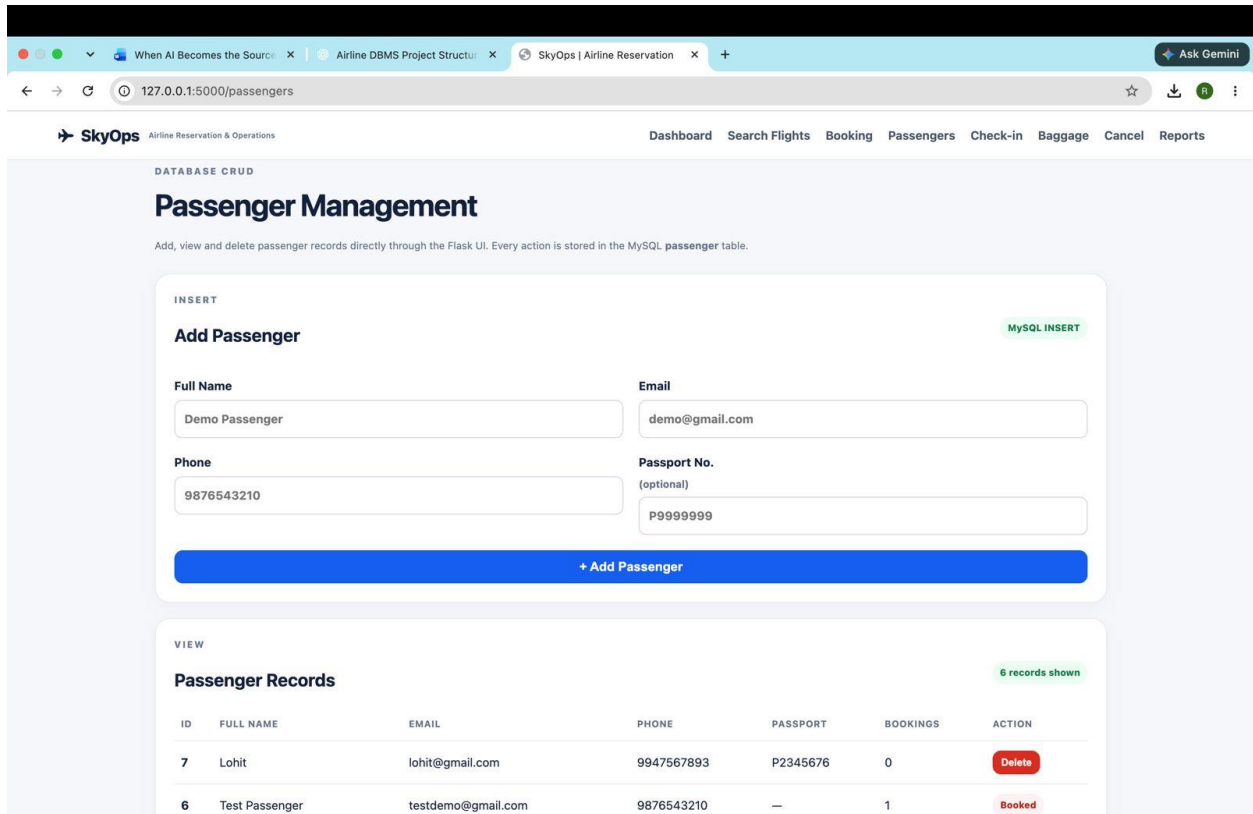


Figure 8: Database and Application Synchronization.

11. IMPLEMENTATION DETAILS

11.1 Technology Stack

Layer	Technology
Backend	Python 3 and Flask
Database	MySQL
Database connectivity	mysql-connector-python
Configuration	python-dotenv and .env
Frontend	HTML, CSS, JavaScript and Jinja templates
Version control	Git and GitHub

The repository's requirements file specifies Flask 3.1.2, mysql-connector-python 9.4.0 and python-dotenv 1.1.1.

11.2 Database Connectivity

The application reads its MySQL connection settings from environment variables.

```
MYSQL_CONFIG = {  
    "host": os.getenv("MYSQL_HOST", "127.0.0.1"),  
    "port": int(os.getenv("MYSQL_PORT", "3306")),  
    "user": os.getenv("MYSQL_USER", "root"),  
    "password": os.getenv("MYSQL_PASSWORD", ""),  
    "database": os.getenv(  
        "MYSQL_DATABASE",  
        "airline_reservation"  
    ),  
}
```

The database connection component opens a MySQL connection, executes parameterized SQL, supports commit and rollback, and closes the connection.

The initialization routine creates the database, applies the schema, ensures indexes and seeds demonstration data.

11.3 Booking Transaction and Integrity

Booking is a multi-step operation. It can involve passenger lookup or insertion, fare validation, seat availability validation, aircraft capacity validation and insertion of booking, ticket and payment records.

A simplified representation of the transaction is:

```
conn.execute("START TRANSACTION")
```

```
# Passenger lookup or insertion  
# Validate fare  
# Validate seat availability  
# Validate aircraft capacity  
# Insert booking, ticket and payment
```

```
conn.commit()
```

If an operation fails, the transaction is rolled back:

```
conn.rollback()
```

This transaction structure helps prevent partial records when a selected seat is unavailable or the aircraft capacity has been reached.

11.4 Reporting Implementation

The application provides operational reports for:

- Flight manifests.
- Flight occupancy.
- Route demand.
- Baggage.
- Cancellations and refunds.
- Revenue.

12. TESTING (TEST CASES AND RESULTS)

12.1 Testing Objective

Testing verifies the expected behaviour of flight search, booking, seat allocation, aircraft capacity validation, baggage validation, check-in, cancellation and reporting.

The repository contains 14 systematic test cases covering these areas, along with passenger handling and the health endpoint.

12.2 Test Cases

ID	Test Case	Expected Result	Documented Status
T01	Search by origin, destination and date	Matching flights are displayed	Implemented
T02	Book an available seat	Booking, ticket and payment are created	Implemented
T03	Select an occupied seat	Transaction is rejected	Implemented
T04	Book beyond aircraft capacity	Transaction is rejected	Implemented
T05	Enter negative baggage values	Validation rejects the input	Implemented
T06	Check in a confirmed booking	Check-in record is created	Implemented
T07	Check in a cancelled booking	Operation is rejected	Implemented
T08	Cancel a confirmed booking	Booking is cancelled and refund is generated	Implemented
T09	Cancel an already cancelled booking	Operation is rejected	Implemented
T10	View flight manifest	Passenger, ticket, seat and check-in status are shown	Implemented
T11	View flight occupancy	Booked count and occupancy percentage are shown	Implemented
T12	View revenue	Paid and refunded amounts are reported	Implemented

T13	Duplicate email in booking flow	Existing passenger is reused or updated in the booking flow	Implemented in booking flow
T14	Health endpoint	JSON status response is returned	Endpoint present

12.3 Testing Results

The repository documents the expected results and implementation status of the test cases. These statuses reflect documented application logic and the project demonstration flow; they should not be interpreted as evidence of a fresh automated test run.

12.4 Verification Note

The project distinguishes documented implementation behaviour from independently executed test evidence. The final submission should include actual screenshots or logs for any tests claimed to have been executed during final verification.

13. CONCLUSION AND FUTURE ENHANCEMENTS

13.1 Conclusion

SkyOps demonstrates how a relational database management system can support airline reservation and flight-operation workflows. The project models 13 connected entities, converts the ER model into a relational schema, normalizes the design to Third Normal Form and applies integrity constraints to preserve consistency.

The Flask application connects database operations to a web interface and demonstrates flight search, booking, passenger management, check-in, baggage, cancellation and reporting functions.

The project also demonstrates SQL as an operational decision-making tool rather than only a storage mechanism. Joins combine related records for flight manifests, aggregate functions calculate occupancy and demand, nested queries identify flights above thresholds, and views provide reusable reporting structures.

Overall, SkyOps provides practical experience in database design, normalization, SQL, integrity enforcement and application-database integration.

13.2 Future Enhancements

Area	Enhancement
------	-------------

Security	Add authentication, role-based authorization, secure secret management and stronger session controls.
Concurrency	Use stronger transaction/isolation strategies and row-level locking for high-contention seat booking.
Validation	Add richer email, phone and passport validation and centralized server-side validation.
Operations	Add flight crew, gate, boarding and delay-management modules.
Analytics	Add dashboards and trend charts for demand, occupancy, revenue and cancellations.
Scalability	Introduce connection pooling, caching and pagination for larger datasets.
Deployment	Containerize the application and database and provide production deployment documentation.
Testing	Add automated unit, integration and database constraint tests with continuous integration execution.

These improvements could make the system more secure, scalable and suitable for more complex airline operational requirements.

14. REFERENCES

SkyOps Project Repository. DBMS-CSE-CORE-Project-18 — Airline Reservation & Flight Operations Management System. GitHub. Available at:
<https://github.com/Rathneshvaran/DBMS-CSE-CORE-Project-18>

SkyOps Project Repository. schema.sql — MySQL DDL and database constraints.

SkyOps Project Repository. docs/DATA_DICTIONARY.md — Table purposes and integrity constraints.

SkyOps Project Repository. docs/NORMALIZATION.md — First, Second and Third Normal Forms and functional dependencies.

SkyOps Project Repository. docs/SQL_REPORTS.sql — Manifest, occupancy, route demand, baggage, cancellation, revenue and view queries.

SkyOps Project Repository. docs/TEST_CASES.md — Systematic test cases.

SkyOps Project Repository. app.py — Flask routes, MySQL connectivity, booking transactions, passenger CRUD and reports.

SkyOps DBMS Final Presentation supplied with the project.

Oracle. MySQL Reference Manual. Available at: <https://dev.mysql.com/doc/>

Pallets Projects. Flask Documentation. Available at: <https://flask.palletsprojects.com/>

Python Software Foundation. Python Documentation. Available at: <https://docs.python.org/3/>

GitHub. GitHub Documentation. Available at: <https://docs.github.com/>

15. CONTRIBUTION OF EACH MEMBER

The project has three members. The following contribution split is aligned with the project's database, application, documentation and presentation deliverables. The wording should be adjusted if the actual task history differs.

Member	Roll Number	Contribution
--------	-------------	--------------

Rathneshvaran.I	25WU0101105	Repository lead and core database/application work; schema and MySQL implementation; seed data; booking and transaction logic; repository maintenance.
Lohit Sitharth.V.O	25WU0101066	DBMS analysis and documentation; ER, schema and normalization preparation; SQL query/report preparation; UI/demo documentation and presentation support.
Sanjay.K	25WU0101122	Testing and validation support; UI/CRUD demonstration; report and presentation preparation; integration and final verification support.
Mrithun Jagan	25WU0101078	SQL query development and testing support; database validation; preparation of test cases and sample outputs; assistance with project documentation and final presentation.

All members contributed to project discussions, integration and final presentation.

16. APPENDIX: GITHUB REPOSITORY LINK

16.1 Project Repository

The complete source code, database schema and supporting DBMS documentation are maintained in the project's GitHub repository.

Repository Link:

<https://github.com/Rathneshvaran/DBMS-CSE-CORE-Project-18>

16.2 Repository Contents

Path	Purpose
app.py	Flask application, MySQL connection, routes, booking logic, CRUD and reports.
schema.sql	Primary MySQL DDL for the 13-table relational database.
requirements.txt	Python dependencies.
docs/DATA_DICTIONARY.md	Data dictionary and integrity rules.
docs/NORMALIZATION.md	Normalization and functional dependencies.
docs/SQL_REPORTS.sql	Operational reporting queries and view.
docs/TEST_CASES.md	Systematic test cases.
templates/	Web interface templates.
static/	Static assets, including stylesheets.

The README documents the technology stack, setup procedure, database tables, demonstration flow and DBMS deliverables.

16.3 Local Application URL

When the Flask application is running locally, it can be accessed at:

<http://127.0.0.1:5000>

This is a local address accessible on the computer running the application. It is not a public deployment URL.

